

MongoDB

WRTN Technologies - Health Check Report

MongoDB Consulting Report

PROJECT

crack-prod Cluster Health Check

PREPARED FOR

WRTN Technologies

Version: 1.0

Date: April 2026

PARTICIPANTS

- Kunhan Choi, Principal Consulting Engineer, MongoDB <han.choi@mongodb.com>
- 김민종, Tech Leader, WRTN Technologies <bob@wrtn.io>
- 주성민, Tech Leader, WRTN Technologies <jake@wrtn.io>
- Doohyun Hwang, Senior Project Manager, MongoDB

Table of Contents

1 Executive Summary	
2 Background	
2.1 서비스 개요	
2.2 Application 환경	
2.3 Cluster 환경	
2.3.1 Cluster Configuration	
2.3.2 Shard Configuration	
2.3.3 Shard Storage Summary	
2.3.4 Databases Summary	
2.3.5 Sharded Collection	
2.3.6 Collection Stats	
2.4 Workload Analysis	
2.4.1 Query Operations — Shard Level	
2.4.2 Query Operations — mongos Level	
2.4.3 CPU Utilization	
2.4.4 Query Targeting	
2.4.5 Scan and Orders	
2.4.6 Cross-Shard Transaction	
3 Goal of the Consulting Engagement	
4 Recommendations	
4.1 Sharding 아키텍처를 재설계하십시오	
4.1.1 현재 상태	
4.1.2 Sharding 설계 부재로 인한 문제	
4.1.3 해소를 위한 선행 조건	
4.1.4 해소 전략	
4.1.5 전략 평가	
4.2 인덱스 및 쿼리를 최적화하십시오	
4.2.1 현재 상태	
4.2.2 즉시 적용 가능한 인덱스 추가 목록	
4.2.3 쿼리 패턴 변경이 필요한 항목	
4.3 불필요한 인덱스를 정리 및 제거하십시오	
4.3.1 현재 상태	
4.3.2 불필요한 인덱스의 유형	
4.3.3 미사용 인덱스 식별 방법	
4.3.4 단계적 인덱스 정리	
4.3.5 인덱스 목록	
4.4 Data Fragmentation을 해소하십시오	
4.4.1 Fragmentation 원인	
4.4.2 적용 방법	
4.5 Read 분산을 위해 Secondary Read를 고려하세요	
4.5.1 현재 상태	
4.5.2 secondaryPreferred 적용 가능 기준	
4.5.3 Secondary Read 적용	
4.6 Mongoose ODM에서 Native Driver로의 전환을 계획하십시오	

4.6.1 현재 상태	
4.6.2 권고 조치	
4.7 Online Archive를 활용한 데이터 티어링을 도입하십시오	
4.7.1 현재 상태	
4.7.2 권고 조치	

5 Other Notes & Questions

5.1 Mongoose ODM에 관하여	
5.2 Unique Index 빌드 실패 및 prepareUnique 활용	
5.3 Index를 활용한 In-Memory Sort 제거	
5.4 Online Archive를 활용한 데이터 티어링	

6 Recommended Follow-up Consulting

6.1 Sharding Architecture Optimization	
6.2 Query Performance Optimization Workshop	
6.3 messages 클러스터 분리 설계	
6.4 Data Lifecycle Management	
6.5 Training Subscription	

Appendix

Appendix1. 미사용 / 중복 인덱스 일람	
----------------------------------	--

이 문서는 뤼튼테크놀로지스(WRTN Technologies)와 진행한 5일간의 컨설팅 인게이지먼트에서 논의된 내용과 권고사항을 요약합니다.

1 Executive Summary

뤼튼테크놀로지스(WRTN Technologies)의 crack-prod[0] 클러스터에 대한 5일간 Health Check를 수행한 결과, 총 7건의 Recommendation을 도출하였습니다. 아래 표에 각 권고사항의 핵심 내용, 권고 조치를 요약하였습니다.

#	권고사항	Priority	현재 상태	권고 조치
4.1	Sharding 아키텍처 재설계	P1	189개 Collection 중 1개(messages)만 샤드됨. 188개 Unsharded Collection의 데이터(5.0TB), 인덱스(880GB), Query 트래픽(99%)이 shard-0에 집중. shard-1/2는 유휴 상태.	0단계: Affinity Group 분석(1주) → 1단계: moveCollection으로 Unsharded Collection 재분배(1-2주) → 2단계: messages를 별도 클러스터로 분리(1-3개월)
4.2	인덱스 및 쿼리 최적화	P1	48시간 동안 14,396회, 누적 101분의 Slow Query 발생. 피크 시 초당 800건의 in-memory sort가 shard-0에 집중. ESR 인덱스 갭, Offset 페이지네이션, COLLSCAN 등 4가지 안티패턴 식별.	6개 인덱스 즉시 추가(chat-summaries, stories, characters, payments, alarms, chats). 4개 쿼리 패턴을 커서 기반 페이지네이션으로 전환.
4.3	불필요한 인덱스 삭제	P1	인덱스가 과도하게 많음. 사용되지 않거나 불필요한 인덱스 정리 필요	인덱스 검토 및 불필요한 인덱스 삭제
4.4	Data Fragmentation 해소	P2	messages Collection의 Data File Fragmentation이 36.2%. Replica Set에서 3 Shard 확장 시 Chunk Migration에 의한 대량 삭제가 원인.	compact 커맨드로 1회성 회수(권장). 향후 Chunk Migration 빈발 시 autoCompact 상시 활성화 검토.
4.5	Secondary Read 도입	P2	모든 Read 쿼리가 Primary 노드로만 전송. 각 Shard의 Secondary 2노드는 Read 처리에 전혀 활용되지 않음.	Connection String을 Primary 전용/Secondary Read 전용 2개로 분리. Stale Read 허용 가능한 쿼리를 secondaryPreferred로 전환. maxStalenessSeconds=90 설정.
4.6	Mongoose → Native Driver 전환	P3	Mongoose v8.15.1 ODM 사용 중. Document Hydration으로 Native Driver 대비 약 2배 느린 Read 성능. Driver 버전이 Mongoose에 종속되어 독립 업그레이드 불가.	단계적으로 Native Driver로 전환. 4.1의 messages 클러스터 분리와 동시 진행 권장. 전환 전 즉시 lean() 옵션 적용으로 Hydration 오버헤드 제거.
4.7	Online Archive 데이터 티어링	P3	채팅 메시지, 활동 로그 등 시간 경과에 따라 접근 빈도가 급감하는 Cold Data가 Hot Data와 동일 클러스터에 저장. messages Collection이 전체 볼륨의 대부분 차지.	Atlas Online Archive로 messages의 90일 이상 경과 데이터를 자동 아카이빙. Partition 필드로 chatId, createdAt 지정.

2 Background

2.1 서비스 개요

뤼튼테크놀로지스(Wrtn Technologies)는 한국의 생성형 AI 기반 콘텐츠 플랫폼 기업으로, 대표 서비스인 Crack을 통해 다양한 AI 캐릭터와 대화할 수 있는 인터랙티브 채팅 서비스를 제공하고 있습니다. 10M+ MAU 규모의 사용자 기반을 보유하고 있으며, 피크 시간대에 50,000명의 동시 사용자 트래픽을 처리합니다.

Crack 서비스는 AI 캐릭터 채팅, 사용자 대화 데이터 저장/관리, 캐릭터 콘텐츠 관리, 사용자 보상(User Reward), 알림(Notification), 개인화(Personalization), 결제(Billing), User Note 등 다양한 워크로드를 MongoDB Atlas 위에서 운영하고 있으며, 한국 시장 외에 일본(Crack Japan), 미국(Crack US) 등 글로벌 시장으로 서비스를 확장 중입니다.

2.2 Application 환경

MongoDB 로그 분석을 통해 확인된 Application 기술 스택은 다음과 같습니다.

항목	값	비고
Runtime	Node.js v20.11.0 (LE, arm64)	LTS 버전
MongoDB Driver	nodejs v6.16.0	Official Node.js Driver
ODM	Mongoose v8.15.1	Community-Supported Library
Container Orchestrator	Kubernetes	Application Pod 운영
OS	Linux arm64 (6.12.68)	ARM 기반 인스턴스
CDC Pipeline	MongoDB Kafka Source Connector v1.15.0	Java Driver v4.7.2 기반, 2개 인스턴스 운영

Application과 CDC 파이프라인이 서로 다른 기술 스택(Node.js vs Java)으로 독립적으로 운영되고 있으며, Kubernetes 환경에서 컨테이너화되어 배포되고 있습니다. CDC 파이프라인은 별도의 Amazon Linux 2023 인스턴스에서도 운영되는 것이 확인되어, 2개의 Kafka Connector가 동작하고 있습니다.

2.3 Cluster 환경

crack-prod 클러스터는 MongoDB Atlas에서 3개의 Shard로 운영되는 Sharded Cluster입니다. 각 Shard는 PSS(Primary-Secondary-Secondary) 구성의 Replica Set으로 이루어져 있습니다.

2.3.1 Cluster Configuration

항목	값
MongoDB Version	8.0.20
Cluster Type	Sharded Cluster (3 Shards)
Cloud Provider	AWS
Disk IOPS	15,000
Auto-Scaling	미사용

MongoDB 최신 버전인 8.0을 사용하고 있습니다. Disk IOPS는 15K입니다.

2.3.2 Shard Configuration

항목	shard-0 (M200)	shard-1 (M140)	shard-2 (M140)
Memory	246.3GB	184.6GB	184.6GB
Cores	64	48	48
WiredTiger Cache	164.0GB	120.0GB	120.0GB
Oplog Max Size	1.1GB	50.0GB	50.0GB
Oplog Size Used	42.4GB	49.7GB	49.9GB
Oplog Duration	1.0 days	3.0 days	3.0 days

Oplog Window가 수십분단위로 줄었던 적이 관찰되었는데 일시적인 상황으로 판단됩니다. 현재는 Oplog Window가 1일로 세팅되어 있습니다.

2.3.3 Shard Storage Summary

Shard	Data Size	Index Size	Storage Size
shard-0	5.0TB	880.0GB	6.2TB
shard-1	2.7TB	137.1GB	2.5TB
shard-2	2.7TB	137.3GB	2.5TB
Total	10.4TB	1.1TB	11.2TB

2.3.4 Databases Summary

DB	COL	Index	Index Size	Data Size	Avg Obj Size	Storage
wrtm-copy	189	645	1.1TB	10.4TB	1.2KB	11.2TB
character-events	4	8	3.5GB	6.9GB	116B	3.3GB
test	78	252	1,008KB	0	0	312KB
Total	273	911	1.1TB	10.4TB	-	11.2TB

wrtm-copy Database는 90% 이상 messages Collection에서 사용되고 있습니다.

2.3.5 Sharded Collection

현재 클러스터에서 유일하게 샤드된 Collection은 wrtm-copy.messages이며, 약 4.3 Billion 건의 Document를 보유하고 있습니다. Shard Key는 userId를 기준으로 Hashed 샤드를 사용합니다.

2.3.6 Collection Stats

2.4 Workload Analysis

2.4.1 Query Operations — Shard Level

48시간 동안의 Shard Primary 노드별 Query 오퍼레이션을 분석한 결과, 극단적인 부하 불균형이 확인되었습니다.

Shard	Peak Query ops/s	Off-peak Query ops/s	비중
shard-0	약 10,000	약 3,000	약 99%
shard-1	약 100	약 0	약 0.5%
shard-2	약 100	약 0	약 0.5%

shard-0이 전체 Query 트래픽의 약 99%를 처리하고 있으며, shard-1과 shard-2는 거의 0에 가까운 수준입니다. 이는 messages Collection을 제외한 나머지 188개 Unsharded Collection이 모두 shard-0에 위치하고 있고, 실제 Application의 읽기 워크로드가 이 Unsharded Collection들에 집중되어 있음을 의미합니다. messages Collection의 샤드된 Query는 userId hashed 키를 통해 3개 Shard에 분배되지만, 그 트래픽 규모가 Unsharded Collection 대비 미미한 것으로 분석됩니다.

쿼리 패턴은 한국 시간 기준으로 뚜렷한 일별 주기(Diurnal Pattern)를 보입니다. 피크 시간대(저녁 18:00–24:00)에 약 10,000 ops/s까지 상승하고, Off-peak 시간대(새벽 03:00–09:00)에는 약 3,000 ops/s까지 하락합니다.

2.4.2 Query Operations — mongos Level

mongos 레벨에서의 Query 오퍼레이션은 48시간 동안 약 200–1,000 ops/s 범위로 관찰되었습니다. 모든 mongos 노드의 쿼리 부하가 균등하게 분산되어 있어, Application의 Connection Pool 구성은 적절한 것으로 판단됩니다. mongos 레벨에서의 수치(최대 약 1,000/s)와 Shard 레벨에서의 수치(최대 약 10,000/s) 사이의 차이는, mongos에서 1개의 쿼리가 Shard에서는 다수의 내부 오퍼레이션으로 확장될 수 있기 때문입니다.

2.4.3 CPU Utilization

Shard	Peak CPU	평균 CPU	비고
shard-0 노드들	약 15%	약 5–10%	다른 샤드에 비해 5배 이상 높음
shard-1 노드들	약 3%	약 1–2%	-
shard-2 노드들	약 3%	약 1–2%	-

현재 CPU 사용률 자체는 Critical 수준이 아닙니다. 그러나 shard-0이 M200(64 Core)임에도 shard-1/2(M140, 48 Core)보다 3–5배 높은 CPU를 사용하고 있다는 점은, shard-0에 워크로드가 극도로 집중되어 있다는 추가 증거입니다.

2.4.4 Query Targeting

Query Targeting은 Scanned(Index) / Scanned Object(Collection)의 지표를 returned로 나눈 것으로 얼마나 효과적으로 쿼리를 수행하는지 알아볼 수 있는 지표입니다.

지표	shard-0	shard-1/2	비고
Scanned Keys / Returned (평균)	5–10	1–3	Shard-0이 3배 이상 높음
Scanned Keys / Returned (peak)	약 22	약 5	-
Scanned Objects / Returned (평균)	5–10	3–5	Shard-0이 3배 이상 높음
Scanned Objects / Returned (peak)	약 60	약 15	-

절대적인 수치를 보서는 매우 높은 수준은 아니지만 Query 수를 감안했을 때 상대적으로 비효율적인 쿼리가 의심되는 상황입니다.

2.4.5 Scan and Orders

Shard	Peak (ops/s)	Off-peak (ops/s)	비중
shard-0	약 800	약 200~400	약 100%
shard-1	약 0	약 0	약 0%
shard-2	약 0	약 0	약 0%

scanAndOrder 지표는 in-memory Sort를 수행한 횟수를 나타냅니다. 간헐적으로 튀는 경우가 아닌 이런 식의 Linear한 형태의 추이는 높은 확률로 비효율적인 쿼리가 많이 발생했음을 나타냅니다. 대부분의 Sort는 Index를 통해 제거할 수 있다는 점을 주목할 필요가 있고 간단하고 자주 수행되는 쿼리 위주를 점검해볼 필요가 있습니다. 또한 shard-0에 집중되는 것을 봤을 때 messages가 아닌 unsharded collection이 유력합니다.

2.4.6 Cross-Shard Transaction

shard-0의 twoPhaseCommitCoordinator 메트릭에서 Cross-Shard Transaction이 빈번하게 발생하고 있음이 확인되었습니다.

메트릭	평균	최대	의미
totalCreated	75.9/s	146.8/s	초당 75~146건의 Cross-Shard Transaction 생성
totalStartedTwoPhaseCommit	0.29/s	0.91/s	실제 2-Phase Commit 진행
totalCommittedTwoPhaseCommit	0.29/s	0.91/s	2-Phase Commit 완료

shard-0에서만 이 메트릭이 발생하고 있어, shard-0이 Cross-Shard Transaction의 Coordinator 역할을 담당하고 있습니다.

totalCreated(초당 75-146건)와 실제 2-Phase Commit(초당 0.29~0.91건) 사이에 큰 차이가 존재합니다. 이는 ~~Transaction Coordinator가 생성되더라도, MongoDB 내부적으로 실제 여러 Shard에 걸친 쓰기가 발생하지 않는 경우 Single-Shard로 최적화되어 2-Phase Commit 프로토콜을 건너뛸 수 있기 때문입니다.~~ 즉, totalCreated의 대부분은 Transaction이 시작 시점에 Coordinator를 생성하지만, 실제 Write 대상이 단일 Shard에 국한되어 2-Phase Commit이 불필요한 케이스로 판단됩니다. 그럼에도 불구하고, 실제 2-Phase Commit이 초당 0.29~0.91건 발생하고 있다는 것은 지속적인 Cross-Shard Transaction 오버헤드가 존재함을 의미합니다.

현재 1개 Collection(messages)만 샤드되어 있으므로, 이 Cross-Shard Transaction은 messages(샤드됨, shard-0/1/2 분산)와 Unsharded Collection(shard-0 집중)을 하나의 Transaction으로 묶어 처리하는 패턴일 가능성이 높습니다.

Cross-Shard Transaction은 Single-Shard Transaction 대비 2-Phase Commit 프로토콜로 인한 추가 latency, 각 Shard에서의 동시 Lock으로 인한 contention 증가, Coordinator(shard-0)에 대한 추가 부하 집중, 실패 시 rollback 비용 증가 등의 오버헤드가 있습니다.

로그 분석을 통해 messages에 update하는 부분이 transaction으로 수행되는 것을 확인하였습니다. 이 워크로드에 대해서 꼭 transaction이 필요한지 점검하고, 가능한 Cross Shard Transaction을 제거하는 것이 권장됩니다.

3 Goal of the Consulting Engagement

3/19에 수행한 미팅에서 Professional Service의 소개와 Consulting Engagement의 방향성을 논의하였습니다. 이 과정에서 가장 부하 및 크기가 큰 서비스인 crack-prod Cluster를 점검을 통해 향후 Consulting의 방향성을 수립하기로 합의하였습니다.

이번 Health Check 및 진단은 다음과 같은 방향으로 진행하였습니다.

항목	내용
클러스터 상태 진단	crack-prod 클러스터의 전반적인 상태를 진단하고, 잠재적 위험 요소를 사전에 식별
Sharding 아키텍처 평가	3-Shard 구성에서 1개 Collection만 샤드된 아키텍처의 적정성을 평가
비용 최적화	Tier 및 Shard 구성의 효율성을 검토
Best Practice 점검	Replication, Index, Query Performance 등 운영 관점의 Best Practice 준수 여부를 점검
아키텍처 개선	향후 서비스 확장을 위한 아키텍처 개선 방향을 제안

Priority Legend

Priority	Meaning
Priority 1	Implement immediately. Not implementing incurs risk of data loss, system unavailability, or other significant problems that may cause outages.
Priority 2	Implement as soon as possible. Significant problems that could affect a production system; consider these issues promptly.
Priority 3	Consider this recommendation. Not critical; may reflect a larger change and may be part of the next revision.

Important: All recommendations in this report should be tested in a pre-production environment before being applied to production.

4 Recommendations

★ Important

본 보고서에 포함된 모든 권고 사항은 운영 환경에 적용하기 전에 반드시 사전 검증 환경(Pre-production)에서 충분히 테스트한 후 적용하시기 바랍니다.

4.1 Sharding 아키텍처를 재설계하십시오 Priority 1

4.1.1 현재 상태

crack-prod 클러스터는 wrtn-copy 데이터베이스에 총 189개의 Collection을 보유하고 있으나, 이 중 messages Collection 1개만 샤드되어 있습니다. 현재 구조를 분석하면, 기존 Replica Set을 Sharded Cluster로 전환하는 과정에서 워크로드 패턴 분석, 데이터 모델링 검토, Shard Key 설계 등의 사전 설계 없이 messages Collection만 샤딩한 것으로 추정됩니다.

Sharded Cluster 도입 시에는 다음의 설계 검토가 선행되어야 합니다.

검토 항목	설명
샤딩 대상 Collection 선정	어떤 Collection을 샤딩할 것인가 (워크로드 분석 기반)
Shard Key 설계	각 Collection의 Shard Key를 어떻게 설계할 것인가 (ESR 원칙, Cardinality, Write Distribution)
Transaction/\$lookup 의존성	Collection 간의 Transaction 및 \$lookup 의존성이 Sharding 후에도 유지 가능한가
Primary Shard 배치	Unsharded Collection의 Primary Shard 배치를 어떻게 할 것인가

이러한 검토 없이 Sharding을 적용하면, 구조적 문제가 발생합니다.

4.1.2 Sharding 설계 부재로 인한 문제

현재 구조에서 발생하고 있는 문제는 크게 두 가지로 분류됩니다.

첫째, shard-0에 대한 워크로드 편중(Hot Shard)입니다. MongoDB Sharded Cluster에서 Unsharded Collection은 해당 데이터베이스의 Primary Shard에만 저장됩니다. 189개 중 1개만 샤드되어 있으므로, 나머지 188개 Collection의 데이터, 인덱스, 그리고 이에 대한 모든 Read/Write 워크로드가 shard-0(Primary Shard) 한 곳에 집중됩니다.

영역	현재 상태	영향
데이터 불균형	shard-0: 5.0TB, 880GB Index / shard-1,2: 각 2.7TB, 137GB Index	shard-0만 M200 Tier 필요, 비대칭 비용 구조
워크로드 집중	shard-0이 전체 Query의 약 99% 처리 (피크 약 10,000 ops/s)	3개 Shard 비용을 지불하지만 1개만 실질 가동
Cache Pressure	인덱스 880GB vs WiredTiger Cache 164GB (5.4배)	인덱스의 18.7%만 Cache 수용, Slow Query 의 90-96%가 Disk I/O

둘째, Cross-Shard Transaction 오버헤드입니다. messages Collection이 3개 Shard에 분산되어 있고, 나머지 Collection은 shard-0에만 존재하는 구조에서, Application이 messages와 다른 Collection을 하나의 Transaction으로 묶어 처리하면 반드시 Cross-Shard Transaction이 발생합니다. 2.4.6절의 Metrics에서 초당 75-146건의 Cross-Shard Transaction이 확인되었으며, 이는 다음의 오버헤드를 수반합니다.

- 2-Phase Commit 프로토콜로 인한 추가 latency
- 각 Shard에서의 동시 Lock으로 인한 contention 증가
- shard-0이 Coordinator 역할을 담당하여 추가 부하 집중
- 실패 시 rollback 비용 증가

이 두 문제는 독립적이지만 동일한 근본 원인(Sharding 설계 부재)에서 기인합니다. Hot Shard는 Unsharded Collection의 배치 문제이고, Cross-Shard Transaction은 샤드된 messages와 Unsharded Collection 간의 데이터 의존성 문제입니다.

4.1.3 해소를 위한 선행 조건

사전에 반드시 데이터 액세스 패턴을 분석해야 합니다.

현재 Application에서 Transaction과 \$lookup이 어떤 Collection 조합으로 사용되고 있는지를 먼저 파악해야 합니다.

moveCollection으로 Unsharded Collection을 다른 Shard로 이동하면, 기존에 Single-Shard에서 처리되던 Transaction이 Cross-Shard Transaction으로 전환될 수 있습니다. 예를 들어, 현재 shard-0에 함께 위치한 chats와 character-chats를 하나의 Transaction으로 처리하고 있다면, chats를 shard-1로 이동하는 순간 이 Transaction은 Cross-Shard Transaction이 되어 2-Phase Commit 오버헤드가 추가됩니다.

\$lookup의 경우, Sharded Cluster에서 \$lookup의 from Collection이 다른 Shard에 위치하면 네트워크 왕복에 따른 비용이 추가됩니다.

따라서 다음의 분석을 선행해야 합니다.

1. Application 코드에서 Multi-document Transaction을 사용하는 모든 코드 경로를 식별하고, 각 Transaction에 포함된 Collection 목록을 정리합니다.

2. \$lookup Aggregation Pipeline에서 참조하는 Collection 조합을 전수 조사합니다.
3. 위 결과를 바탕으로, 동일 Shard에 함께 위치해야 하는 Collection 그룹(Affinity Group)을 정의합니다.

이 Affinity Group 정보가 확보되어야 moveCollection이나 클러스터 분리 시 어떤 Collection을 함께 이동해야 하는지, 어떤 Transaction/lookup 패턴을 사전에 제거하거나 재설계해야 하는지를 판단할 수 있습니다.

4.1.4 해소 전략

선행 조건이 충족된 후 적용할 수 있는 2가지 전략을 제시합니다.

Strategy A: moveCollection을 활용한 Unsharded Collection 재분배

MongoDB 8.0에서 도입된 moveCollection 커맨드를 사용하여 Unsharded Collection을 다른 Shard로 이동하는 전략입니다. Application 코드 변경 없이 온라인으로 수행 가능합니다. 188개 Unsharded Collection 중 사이즈가 큰 것들을 shard-1/2로 분배하여 인덱스 사이즈를 shard당 약 300GB 수준으로 균등화하면, shard-0의 M200을 더 낮은 Tier로 Scale-down할 수 있습니다. 다만 Affinity Group에 속한 Collection들은 반드시 같은 Shard에 함께 이동해야 합니다.

Strategy B: messages Collection을 별도 클러스터로 분리

messages Collection(4.3B 문서, 약 8TB)을 별도의 Sharded Cluster로 분리하고, crack-prod를 Replica Set으로 전환하는 전략입니다. Hot Shard 문제를 원천적으로 해결하며, 비용에 대한 부담을 크게 줄일 수 있는 장점이 있습니다. 다만 messages와 다른 Collection 간의 \$lookup 및 Transaction 의존성을 사전에 제거해야 합니다.

4.1.5 전략 평가

평가 기준	Strategy A (moveCollection)	Strategy B (클러스터 분리)
효과	중	상
구현 난이도	하	상
코드 변경	없음	필요
비용 절감	중	상 (1 ReplicaSet + 1 Sharded Cluster로 구성하지만 현재와 같은 유휴 샤드를 제거)
선행 조건	Affinity Group 정의 필수	\$lookup/Tx 의존성 제거 모델링 전환
리스크	최소 (Affinity 준수 시), 비용 절감 효과 제한	중 (\$lookup/Tx 검증), 구현 난이도 높음

4.2 인덱스 및 쿼리를 최적화하십시오 Priority 1

4.2.1 현재 상태

48시간 동안 14,396회, 누적 101분의 Slow Query가 발생하고 있으며, 피크 시 초당 800건의 in-memory sort(Scan and Orders)가 shard-0에서만 집중적으로 발생하고 있습니다. 645개의 인덱스가 존재하지만, 실제 쿼리 패턴과 체계적으로 불일치하고 있습니다.

분석 결과 반복적으로 관찰되는 4가지 안티패턴이 식별되었습니다.

패턴	설명	영향도
ESR 인덱스 갭	복합 인덱스의 중간 필드가 쿼리에서 사용되지 않아, 인덱스의 정렬 순서를 활용할 수 없음	최고
Offset 페이지네이션	skip()을 사용하여 대량의 Document를 건너뛰는 패턴. 페이지가 깊어질수록 성능이 선형적으로 악화	높음
\$ne/\$exists/\$nin 연산자	부정 조건 연산자는 인덱스를 효율적으로 활용할 수 없어 광범위한 스캔을 유발	중-높음
COLLSCAN 집계	적절한 인덱스가 없어 전체 Collection을 스캔하는 Aggregation Pipeline	높음

4.2.2 즉시 적용 가능한 인덱스 추가 목록

다음 6개의 인덱스를 즉시 추가하면, 48시간 기준 가장 빈번하고 심각한 Slow Query를 해소할 수 있습니다.

Collection	권고 인덱스	해소되는 문제	예상 효과
chat-summaries	<code>{chatId:1, createdAt:-1}</code>	ESR 갭, in-memory sort 2,869회/48h	전체 hasSortStage의 70% 해소
stories	<code>{moderationStatus:1, gracePeriodExpiresAt:1}</code>	매분 COLLSCAN 684K건	docsExamined 684K → 0-1
characters	<code>{moderationStatus:1, gracePeriodExpiresAt:1}</code>	매분 COLLSCAN 56K건, max 16초	docsExamined 56K → 0-1
payments	<code>{userId:1, createdAt:-1}</code>	ESR 갭 + Plan Cache Thrashing	hasSortStage 제거, replanned 해소
alarms	<code>{userId:1, type:1, _id:-1}</code>	ESR 갭 313회/48h	type 필터링 인덱스 완결
chats	<code>{userId:1, isDeleted:1, pinnedAt:-1}</code>	\$exists + ESR 갭, avg 706ms	userId 우선 필터링

chat-summaries

현재 이 Collection에서는 chatId로 조회 후 createdAt 역순 정렬을 수행하는 쿼리가 48시간 동안 2,869회 실행되고 있으며, 평균 311ms, 최대 12,755ms의 응답 시간을 보이고 있습니다.

현재 쿼리

```
db.getCollection("chat-summaries")
  .find({ chatId: ObjectId("...") })
  .sort({ createdAt: -1 })
  .limit(1)
```

현재 실행 플랜은 인덱스 `{ chatId: 1, isOutdated: 1, createdAt: -1 }`를 사용하고 있으나, 쿼리가 isOutdated 필드를 사용하지 않아 createdAt의 인덱스 정렬을 활용할 수 없는 상태입니다(ESR 인덱스 갭). 결과적으로 chatId에 해당하는 모든 키(6,075건)를 스캔한 뒤 in-memory sort를 수행합니다. replanned: true 상태가 반복 발생하여(expected 2 works but took 20) Plan Cache도 불안정합니다.

항목	현재 상태	인덱스 적용 후
사용 인덱스	<code>{chatId:1, isOutdated:1, createdAt:-1}</code>	<code>{chatId:1, createdAt:-1}</code>
hasSortStage	true (in-memory sort)	false (인덱스 정렬)
keysExamined	6,075	1
docsExamined	1	1
replanned	true	false

stories 및 characters

stories와 characters 두 Collection에서 moderationStatus와 gracePeriodExpiresAt 조건으로 필터링하는 Aggregation Pipeline이 매분 실행되고 있습니다. 적절한 인덱스가 없어 매분 전체 Collection을 스캔하며, 두 Collection 합산으로 하루 약 10.6억 건의 불필요한 Document Scan이 발생합니다.

현재 쿼리 (stories)

```
db.getCollection("stories").aggregate([
  { $match: {
    moderationStatus: { $in: ["grace_period_with_image_censor", "grace_period"]
  },
  gracePeriodExpiresAt: { $lte: new Date() }
}},
  { $lookup: { from: "audits", ... }},
  { $lookup: { from: "story-snapshots", ... }},
  ...
])
```

항목	stories 현재	characters 현재	인덱스 적용 후 (공통)
실행 플랜	COLLSCAN	COLLSCAN	IXSCAN
docsExamined	684,161	56,496	0-1
nreturned	0-1	0	0-1
Avg Docs (48h)	-	435,685 (\$lookup 포함)	최소화

권고 인덱스 `{ moderationStatus: 1, gracePeriodExpiresAt: 1 }` 를 두 Collection 모두에 추가하면 COLLSCAN이 완전히 제거됩니다. \$lookup 대상 collection에 인덱스는 정상적으로 구성되어 있습니다.

payments

payments Collection에서 userId로 조회 후 createdAt 역순 정렬을 수행하는 쿼리입니다.

현재 쿼리

```
db.getCollection("payments")
  .find({ userId: ObjectId("...") })
  .sort({ createdAt: -1 })
  .limit(10)
```

현재 인덱스 { userId: 1, type: 1, createdAt: 1 } 에서 type 필드가 쿼리에 사용되지 않아 ESR 갭이 발생합니다. 추가로 Plan Cache Thrashing(캐시된 플랜이 10배 비효율적임을 감지하고 재계획)이 매번 발생하여 불안정한 성능 변동을 야기합니다.

항목	현재 상태	인덱스 적용 후
사용 인덱스	{userId:1, type:1, createdAt:1}	{userId:1, createdAt:-1}
hasSortStage	true	false
keysExamined	361	10
replanned	true (expected 33 works but took 330)	false

alarms

alarms Collection에서 userId와 type으로 필터링 후 _id 역순 정렬을 수행하는 쿼리입니다.

현재 쿼리

```
db.getCollection("alarms")
  .find({ type: "like", userId: ObjectId("...") })
  .sort({ _id: -1 })
  .limit(21)
```

현재 인덱스 { userId: 1, _id: -1 } 에 type이 포함되어 있지 않아, userId 매칭 후 모든 Document를 읽어 type을 확인해야 합니다. 277건을 검사하고 0건을 반환하는 비효율이 발생합니다.

항목	현재 상태	인덱스 적용 후
사용 인덱스	{userId:1, _id:-1}	{userId:1, type:1, _id:-1}
keysExamined	277	0-21
docsExamined	277	0-21

chats

chats Collection에서 userId, isDeleted, pinnedAt 조건으로 조회 후 pinnedAt 역순 정렬을 수행하는 쿼리입니다.

현재 쿼리

```
db.getCollection("chats")
  .find({
    isDeleted: false,
    pinnedAt: { $exists: true },
    userId: ObjectId("...")
  })
  .sort({ pinnedAt: -1 })
```

현재 인덱스 { isDeleted: 1, userId: 1, pinnedAt: -1, messagedAt: -1, createdAt: -1, story.type: 1 } 에서 Selectivity가 낮은 isDeleted(대부분 false)가 userId보다 앞에 위치하여, 인덱스의 첫 번째 필드에서 대부분의 Document가 매칭되어 필터링 효과가 미미합니다.

항목	현재 상태	인덱스 적용 후
사용 인덱스	{isDeleted:1, userId:1, pinnedAt:-1, ...}	{userId:1, isDeleted:1, pinnedAt:-1}
keysExamined	278	0-수건
docsExamined	278	0-수건
핵심 변경	Selectivity 낮은 isDeleted 선행	Selectivity 높은 userId 선행

4.2.3 쿼리 패턴 변경이 필요한 항목

다음 항목들은 인덱스 추가만으로는 해결되지 않으며, Application 코드의 쿼리 패턴 자체를 변경해야 합니다.

Collection	현재 패턴	권고 변경	해소되는 문제
character-messages, messages	<code>_id: { \$gt }</code> 커서	createdAt 커서로 전환	인덱스 미포함 필드로 인한 Document Fetch
notification-settings	<code>skip(12000).limit(4000)</code>	커서 기반 + \$ne를 긍정 조건 전환	1.86M keys 스캔, avg 2,745ms
cracker-histories	<code>skip(1620)</code>	커서 기반 전환	max 13초, offset 안티패턴
stories	<code>\$skip(38300)</code>	firstPublicAt 커서 기반 전환	38K건 불필요 스캔

character-messages 및 messages

두 Collection에서 `_id`를 커서로 사용하여 페이지네이션을 수행하는 쿼리입니다. `_id` 조건이 사용 중인 인덱스에 포함되어 있지 않아, 인덱스로 `userId/chatId/isDeleted`를 필터링한 후 각 Document를 Disk에서 읽어 `_id` 조건을 하나씩 확인해야 합니다. 261건을 읽고 1건만 반환하는 비효율이 발생하며, `bytesRead`가 9.5MB로 96%가 Disk I/O입니다.

현재 쿼리

```
db.getCollection("character-messages").find({
  userId: ObjectId("..."),
  chatId: ObjectId("..."),
  isDeleted: false,
  _id: { $gt: ObjectId("...") }
}).sort({ createdAt: 1 }).limit(21)
```

권고 변경 후

```
db.getCollection("character-messages").find({
  userId: ObjectId("..."),
  chatId: ObjectId("..."),
  isDeleted: false,
  createdAt: { $gt: ISODate("...") }
}).sort({ createdAt: 1 }).limit(21)
```

`createdAt`은 이미 인덱스 `{ userId: 1, chatId: 1, isDeleted: 1, createdAt: -1 }`에 포함되어 있으므로, `_id` 대신 `createdAt`을 커서로 사용하면 인덱스만으로 필터링+정렬+페이지네이션이 완결됩니다.

notification-settings

이 쿼리는 세 가지 문제가 복합적으로 작용하는 가장 심각한 Slow Query입니다.

현재 쿼리

```
db.getCollection("notification-settings")
  .find({ attendance: { $ne: false } })
  .sort({ _id: 1 })
  .skip(12000)
  .limit(4000)
```

권고 변경 후

```
db.getCollection("notification-settings")
  .find({
    attendance: true,
    _id: { $gt: ObjectId("<last_id>") }
  })
  .sort({ _id: 1 })
  .limit(4000)
```

문제 요소	설명
\$ne 연산자	인덱스를 효율적으로 활용할 수 없어 _id 기본 인덱스로 전체 Collection을 스캔
skip(12000)	12,000건을 건너뛰기 위해 해당 건수를 모두 읽어야 함
복합 효과	두 가지가 결합되어 평균 186만 건의 인덱스 키를 스캔

권고 사항으로, 커서 기반 전환과 함께 \$ne: false 조건이 attendance: true와 동일한 의미인지 비즈니스 로직을 확인하십시오. 추가 권고 인덱스는 { attendance: 1, _id: 1 } 입니다.

cracker-histories

skip(1620)으로 인해 1,630건을 읽어 10건만 반환하는 전형적인 offset 안티패턴입니다. \$ne: "expiration" 조건도 인덱스에서 처리되지 않아 Document 레벨에서 필터링됩니다.

현재 쿼리

```
db.getCollection("cracker-histories").find({
  isConsumed: true,
  kind: { $ne: "expiration" },
  userId: ObjectId("...")
}).sort({ createdAt: -1 }).skip(1620).limit(10)
```

권고 변경 후

```
db.getCollection("cracker-histories").find({
  isConsumed: true,
  kind: { $ne: "expiration" },
  userId: ObjectId("..."),
  createdAt: { $lt: ISODate("<last>") }
}).sort({ createdAt: -1 }).limit(10)
```

createdAt 커서 기반으로 전환하면 skip으로 인한 불필요한 스캔이 제거됩니다.

stories Aggregation

\$skip: 38300으로 인해 38,320건의 인덱스 키를 스캔해야 하며, 이후 \$lookup으로 audits Collection을 조인하므로 스캔량에 비례하여 조인 비용도 증가합니다.

현재 쿼리

```
db.getCollection("stories").aggregate([
  { $match: {
    countryCode: "KR",
    deletedAt: { $exists: false },
    status: "active",
    visibility: "public"
  } },
  { $sort: { firstPublicAt: -1 } },
  { $skip: 38300 },
  { $limit: 20 },
  { $lookup: { from: "audits", ... } }
])
```

권고 변경 후

```
db.getCollection("stories").aggregate([
  { $match: {
    countryCode: "KR",
    deletedAt: { $exists: false },
    status: "active",
    visibility: "public",
    firstPublicAt: { $lt: ISODate("<last>") }
  } },
  { $sort: { firstPublicAt: -1 } },
  { $limit: 20 },
  { $lookup: { from: "audits", ... } }
])
```

\$skip을 제거하고 \$match에 firstPublicAt: { \$lt: ISODate("") } 조건을 추가하여 커서 기반 페이지네이션으로 전환합니다.

4.3 불필요한 인덱스를 정리 및 제거하십시오

Priority 1

4.3.1 현재 상태

현재 클러스터 전체에 645개의 인덱스가 존재합니다. 이는 Collection당 평균 인덱스 수가 과도하게 높은 수준이며, Slow Query 분석 결과 실제 쿼리 패턴과 체계적으로 불일치하고 있습니다. 사용되지 않는 인덱스는 단순히 공간만 낭비하는 것이 아니라, 모든 Write 연산의 성능을 저하시키고, WiredTiger Cache를 점유하며, Query Planner의 Plan 선택을 복잡하게 만듭니다. Atlas의 Performance Advisor와 \$indexStats를 활용하여 미사용 인덱스를 식별하고, 단계적으로 정리할 것을 권장합니다.

불필요한 인덱스가 시스템에 미치는 영향은 다음과 같습니다.

영향	메커니즘
Write 성능 저하	모든 Insert/Update/Delete 시 해당 Collection의 전체 인덱스 B-Tree를 갱신해야 함. 인덱스가 많을수록 Write Latency가 선형적으로 증가
Storage 낭비	사용되지 않는 인덱스가 Disk 공간을 점유. 3노드 Replica Set에서는 인덱스 크기의 3배가 소비됨
WiredTiger Cache 압박	인덱스 Page가 Cache에 로드되어 유효 Cache 용량을 감소시킴. Working Set이 Cache를 초과하면 Disk I/O가 급증
Query Planner 비효율	후보 인덱스가 많아질수록 Plan 평가에 소요되는 시간이 증가하고, 최적이지 않은 Plan이 선택될 확률이 높아짐
Plan Cache Thrashing	replanned 현상(payments, chat-summaries)의 근본 원인 중 하나. 유사한 인덱스가 다수 존재하면 Planner가 Plan 간 경합을 반복

4.3.2 불필요한 인덱스의 유형

645개의 인덱스 중 정리 대상이 될 수 있는 유형은 다음과 같습니다.

유형	설명	식별 방법
미사용 인덱스	일정 기간 동안 어떤 쿼리에서도 사용되지 않은 인덱스	\$indexStats에서 accesses.ops = 0
중복 인덱스	다른 인덱스의 prefix와 완전히 동일한 인덱스. 예: {a:1}은 {a:1, b:1}이 존재하면 중복	인덱스 키 패턴 비교
유사 인덱스	필드 구성이 거의 동일하고 순서만 다르거나 1개 필드만 차이 나는 인덱스. 통합 가능성이 높음	인덱스 키 패턴 비교
ESR 갭 유발 인덱스	쿼리에서 사용하지 않는 중간 필드를 포함하여 Sort/Range 최적화를 방해하는 인덱스	Slow Query의 hasSortStage 분석

4.3.3 미사용 인덱스 식별 방법

\$indexStats Aggregation

\$indexStats는 mongod 재시작 이후 각 인덱스가 사용된 횟수를 추적합니다. 충분한 기간(최소 2주 이상, 모든 업무 사이클을 포함)이 경과한 후 조회하면 미사용 인덱스를 식별할 수 있습니다.

특정 Collection의 인덱스 사용 통계 조회

```
db.collection.aggregate([
  { $indexStats: {} }
])

// 결과 예시
{
  "name": "userId_1_type_1_createdAt_1",
  "accesses": {
    "ops": 0,           // 사용 횟수 = 0 -> 미사용 후보
    "since": ISODate("2025-03-01T00:00:00Z")
  }
}
```

Sharded Cluster에서는 각 Shard마다 별도의 통계가 수집되므로, 모든 Shard에서 ops = 0인 인덱스만 미사용으로 판단해야 합니다. 한 Shard에서라도 사용되고 있다면 삭제해서는 안 됩니다. Secondary Read를 활용하는 경우는 Replica 노드에서 위 명령을 수행하여야 합니다.

4.3.4 단계적 인덱스 정리

인덱스 삭제는 되돌리기 어려운 작업이므로(재생성 시 빌드 시간과 리소스 소요), 반드시 단계적으로 진행해야 합니다.

Step 1. 현황 수집

모든 Collection의 \$indexStats를 수집하고, 인덱스별 사용 횟수와 마지막 사용 시점을 정리합니다. 최소 2주 이상의 관찰 기간이 필요하며, 월간/분기간 배치 작업에서만 사용되는 인덱스를 오판하지 않도록 주의하십시오.

전체 Collection 인덱스 통계 일괄 수집

```
db.getCollectionNames().forEach(function(coll) {
  print("=== " + coll + " ===");
  db[coll].aggregate([{$indexStats: {}}]).forEach(function(idx) {
    print(idx.name + " | ops: " + idx.accesses.ops
      + " | since: " + idx.accesses.since);
  });
});
```

Step 2. 삭제 후보 선정

다음 조건을 모두 충족하는 인덱스를 삭제 후보로 선정합니다.

조건	설명
\$indexStats ops = 0	충분한 관찰 기간 동안 한 번도 사용되지 않음
모든 Shard에서 미사용	Sharded Collection의 경우 전체 Shard에서 ops = 0 확인
_id 인덱스가 아닐 것	_id 인덱스는 삭제 불가
Shard Key 인덱스가 아닐 것	Shard Key를 포함하는 인덱스는 Chunk Migration에 필수
TTL 인덱스가 아닐 것	TTL 인덱스는 쿼리에 사용되지 않더라도 자동 삭제 기능을 수행

Step 3. Hidden Index로 전환 후 삭제

인덱스를 바로 삭제하는 것은 위험합니다. 재생성에 대규모 Collection의 경우 수 분~수십 분이 소요되며, 그 동안 해당 인덱스에 의존하던 쿼리의 성능이 급격히 저하될 수 있습니다. 따라서 반드시 Hidden Index 단계를 거쳐 안전하게 삭제할 것을 권장합니다.

Hidden Index는 Query Planner에서 완전히 제외되지만, Write 시에는 여전히 갱신됩니다. 즉, "이 인덱스가 없어도 Read가 정상인가?"를 실서비스에서 검증할 수 있되, 문제가 발생하면 unhideIndex 한 줄로 즉시 복원할 수 있습니다.

Step 3-1. Hidden으로 전환

```

db.collection.hideIndex("index_name")

// Step 3-2. 최소 1~2주 운영하며 모니터링
//   - Slow Query 증가 여부 확인
//   - Atlas Performance Advisor에 새로운 권고가 나타나는지 확인
//   - 해당 Collection의 평균 응답 시간 변화 확인

// 문제 발생 시: 즉시 복원 (인덱스 재생성 불필요, 즉각 반영)
db.collection.unhideIndex("index_name")

// 문제 없음 확인 후: 인덱스 삭제
db.collection.dropIndex("index_name")

```

단계	소요 시간	Read 영향	Write 영향	롤백
hideIndex	즉시	해당 인덱스 미사용	여전히 갱신됨 (변화 없음)	unhideIndex 즉시 복원
dropIndex	즉시	해당 인덱스 미사용	갱신 대상에서 제거 (Write 개선)	createIndex 재생성 필요 (수 분~수십 분)

이 방식의 핵심은 hideIndex → 검증 → dropIndex 순서를 반드시 지키는 것입니다. dropIndex를 바로 실행하면 문제 발생 시 재생성까지의 공백 기간 동안 쿼리 성능이 급락할 수 있지만, hideIndex를 먼저 적용하면 unhideIndex 한 줄로 밀리초 이내에 원래 상태로 복원할 수 있습니다.

항목	설명
실행 시점	hideIndex는 언제든지 가능, dropIndex는 Off-peak 시간대 권장
검증 기간	최소 1~2주, 월간/분기간 배치가 있는 경우 해당 사이클을 포함할 것
Atlas 환경	Hidden Index는 Atlas M10+ Dedicated Cluster에서 지원
Sharded Cluster	각 Shard에서 개별적으로 hideIndex/dropIndex 불필요. mongos를 통해 실행하면 전체 Shard에 자동 적용

4.3.5 인덱스 목록

Appendix로 정리하였습니다.

4.4 Data Fragmentation을 해소하십시오

Priority 2

4.4.1 Fragmentation 원인

WiredTiger는 B-Tree 기반으로 데이터를 leaf page 단위로 저장하며, Document가 Delete되면 해당 page 블록이 checkpoint 시점에 "free block"으로 반환됩니다. 이 free block들이 data file 전체에 흩어져 분포하면 이것이 fragmentation입니다.

messages Collection의 36.2% fragmentation은 Replica Set에서 3개 Shard로 확장하는 과정에서 발생한 Chunk Migration이 원인입니다. 기존 1개 Replica Set에 집중되어 있던 데이터가 3개 Shard로 재분배되면서, donor shard에서 대량의 Chunk가 recipient shard로 이동하였고, 이동이 완료된 Chunk의 데이터는 donor shard에서 물리적으로 삭제됩니다. 이 대량 삭제가 data file 전체에 걸쳐 free block을 남기면서 fragmentation이 발생한 것입니다.

단계	메커니즘	결과
1. Shard 확장	1 RS → 3 Shard로 확장, Balancer가 Chunk 재분배 시작	donor shard에서 대량의 Chunk가 이동 대상으로 선정
2. Chunk Migration	선정된 Chunk의 Document가 recipient shard로 복사	데이터 중복 상태 발생
3. Donor 측 삭제	Migration 완료 후 donor shard에서 해당 Chunk의 Document가 물리 삭제됨	data file 곳곳에 대규모 free block 발생
4. Free block 누적	삭제된 Chunk가 B-Tree 전역에 분산되어 있어 free block이 흩어짐	36.2% fragmentation

Shard 확장이 완료된 이후 새로운 Write가 일부 free block을 재활용하면서 fragmentation이 점진적으로 낮아지고 있으나, messages Collection의 절대적인 크기를 고려하면 자연 해소에 의존하기보다 Compact를 통해 적극적으로 회수하는 것이 권장됩니다.

4.4.2 적용 방법

Shard 확장에 의한 fragmentation은 1회성 이벤트이므로, 대응 방법도 상황에 맞게 선택할 수 있습니다. 즉시 회수가 목적이라면 compact 커맨드를, 향후 Balancer에 의한 Chunk Migration까지 대비한 지속적 관리가 목적이라면 autoCompact를 권장합니다.

Option A: compact 커맨드 (1회성, 권장)

compact 커맨드는 특정 Collection을 대상으로 1회 실행하여 fragmentation을 해소합니다. Shard 확장이라는 1회성 이벤트에 대한 대응으로 가장 적합하며, 실행 전 dryRun 옵션으로 회수 가능한 공간을 사전 확인할 수 있습니다.

자세한 How to 문서는 [How to Use the Compact\(\) Command in Atlas](https://www.mongodb.com/docs/atlas/reference/faq/compact/) (<https://www.mongodb.com/docs/atlas/reference/faq/compact/>) 를 참고하십시오.

Option B: autoCompact (지속적 관리)

향후에도 Balancer에 의한 Chunk Migration이 빈번하게 발생할 것으로 예상된다면, autoCompact를 상시 활성화하여 지속적으로 fragmentation을 회수하는 방안도 고려할 수 있습니다. MongoDB 8.0 이상에서만 사용 가능하며, Atlas에서는 autoCompact Built-in Role이 별도로 필요합니다.

자세한 How to 문서는 [Can I use the autoCompact command in Atlas?](#) 를 참고하십시오.

4.5 Read 분산을 위해 Secondary Read를 고려하세요 Priority 2

4.5.1 현재 상태

현재 Application의 모든 Read 쿼리가 Primary 노드로만 전송되고 있습니다. 각 Shard는 3노드 Replica Set으로 구성되어 있으나, Secondary 노드는 Read 처리에 전혀 활용되지 않고 있어 Primary에 부하가 집중되고 있습니다. 강한 일관성이 필요하지 않은 Read 쿼리에 readPreference: "secondaryPreferred"를 적용하면 Primary의 읽기 부하를 상당 부분 Secondary로 분산할 수 있습니다.

모든 Read를 일괄적으로 Secondary로 전환하는 것이 아니라, 쿼리의 일관성 요구 수준에 따라 분류하여 적용해야 합니다. 아래 기준에 따라 분류하십시오.

4.5.2 secondaryPreferred 적용 가능 기준

다음 조건을 모두 충족하는 Read 쿼리는 secondaryPreferred 적용이 적합합니다.

조건	설명
Stale Read 허용	수 밀리초~수 초의 Replication Lag으로 인해 최신 데이터가 아닌 약간 이전 시점의 데이터가 반환되어도 비즈니스 로직에 영향이 없는 경우
Read-only 컨텍스트	조회 결과를 기반으로 즉시 Write 의사결정을 내리지 않는 경우 (예: 목록 조회, 검색, 통계 대시보드)
비동기 생성 데이터	데이터 자체가 비동기로 생성되거나 주기적으로 갱신되어, Read 시점의 미세한 지연이 사용자 경험에 영향을 주지 않는 경우

다음 조건 중 하나라도 해당하는 Read 쿼리는 반드시 Primary에서 읽어야 합니다.

조건	설명
Read Your Own Write	Write 직후 동일 데이터를 Read하는 패턴. Secondary Lag으로 인해 방금 쓴 데이터가 조회되지 않을 수 있음
Transaction 내부 Read	Multi-document Transaction은 readConcern: "snapshot"으로 동작하며, Session 일관성을 보장하기 위해 Primary 필수
금융/결제 관련	Stale Read가 금전적 오류나 중복 처리를 유발할 수 있는 경우
조회 결과 기반 즉시 Write	조회 결과의 정확성이 후속 Write의 정확성에 직접 영향을 미치는 경우 (예: 재고 확인 후 차감, 잔액 확인 후 결제)
Batch 처리의 정확성 요구	전체 데이터를 순회하며 상태 기반으로 처리하는 배치 작업으로, 누락이나 중복이 허용되지 않는 경우

4.5.3 Secondary Read 적용

개별 쿼리마다 readPreference를 지정하는 방식은 코드 변경 범위가 넓고 유지보수가 어렵습니다. 대신 Connection String을 2개로 분리하여, Application 내에서 용도에 따라 적절한 Connection을 선택하는 방식을 권장합니다.

Connection String 분리

```
# Primary 전용 (기존 유지, 기본값)
mongodb+srv://user:pass@cluster.mongodb.net/dbname
?readPreference=primary

# Secondary Read 전용 (신규 추가)
mongodb+srv://user:pass@cluster.mongodb.net/dbname
?readPreference=secondaryPreferred
&maxStalenessSeconds=90
```

기존 Connection을 그대로 유지하고, Secondary Read용 Connection을 하나 추가합니다. 4.5.2의 분류 기준에 따라 각 서비스/모듈이 적절한 Connection을 선택하면 됩니다.

Connection 2개 생성 및 사용 예

```
const primaryClient = new MongoClient(PRIMARY_URI); // 기존 유지
const secondaryClient = new MongoClient(SECONDARY_URI); // 신규 추가

const primaryDb = primaryClient.db('dbname');
const secondaryDb = secondaryClient.db('dbname');

// Primary Read가 필요한 로직 (기존 코드 변경 없음)
const payment = await primaryDb.collection('payments').findOne({ ... });

// Secondary Read 적용 대상 로직 (db 참조만 변경)
const chatList = await secondaryDb.collection('chats').find({ ... }).toArray();
```

이 방식의 장점은 다음과 같습니다.

장점	설명
최소 코드 변경	개별 쿼리를 수정하지 않고, 서비스/모듈 단위로 db 참조만 교체하면 됨
명확한 분리	Connection 자체가 용도별로 분리되어 있어, 어떤 쿼리가 Secondary로 가는지 아키텍처 레벨에서 명확
롤백 용이	문제 발생 시 Secondary Connection의 readPreference를 primary로 변경하면 즉시 원복 가능
Connection Pool 독립	Primary와 Secondary가 별도 Pool을 사용하므로, Secondary Read 증가가 Primary Connection Pool에 영향을 주지 않음

Secondary Read Connection에 maxStalenessSeconds를 설정하면, 지정된 시간 이상 Replication이 지연된 Secondary는 Read 대상에서 자동으로 제외됩니다. 최소값은 90초이며, Atlas 환경에서 일반적인 Replication Lag은 수 밀리초 수준이므로 90초로 설정해도 대부분의 경우 Secondary에서 정상적으로 Read가 처리됩니다.

4.6 Mongoose ODM에서 Native Driver로의 전환을 계획하십시오

Priority 3

4.6.1 현재 상태

현재 WRTN 엔지니어링 팀은 Node.js 환경에서 Mongoose v8.15.1 ODM을 사용하고 있습니다. Mongoose는 개발 편의성을 제공하지만, 프로덕션 환경에서 성능 오버헤드, Driver 종속성, Support 범위 제외라는 세 가지 단점이 있습니다. 상세 내용은 5.1절을 참조하십시오.

4.6.2 권고 조치

단계적으로 MongoDB Native Driver로 전환하되, 4.1절에서 권고한 messages Collection의 별도 클러스터 분리 (Strategy B)와 동시에 진행하면 자연스러운 전환 시점이 됩니다. 전환 이전이라도 Read-only 쿼리에 lean() 옵션을 즉시 적용하면 Document Hydration 오버헤드가 제거되어 Read 성능 개선 효과를 얻을 수 있습니다.

4.7 Online Archive를 활용한 데이터 티어링을 도입하십시오

Priority 3

4.7.1 현재 상태

WRTN 서비스 특성상 채팅 메시지, 사용자 활동 로그, 결제 스냅샷 등 시간이 지남에 따라 접근 빈도가 급격히 낮아지는 데이터가 지속적으로 누적되고 있습니다. 이러한 Cold Data가 Hot Data와 동일한 클러스터에 저장되면 Working Set 크기가 증가하여 RAM 효율이 떨어지고, Index 크기가 비대해지며, 백업 시간과 Storage 비용이 불필요하게 증가합니다.

4.7.2 권고 조치

Atlas Online Archive로 messages Collection의 createdAt 필드 기준 90일 이상 경과한 데이터를 자동 아카이빙하십시오. Partition 필드는 chatId와 createdAt을 지정하면 특정 채팅방의 과거 메시지 조회 시 최적의 성능을 기대할 수 있습니다. 아카이빙된 데이터는 Atlas Data Federation을 통해 단일 Endpoint에서 클러스터 데이터와 함께 쿼리할 수 있으므로, Application 코드 변경 없이 데이터 라이프사이클을 관리할 수 있습니다. 상세 내용은 5.4절을 참조하십시오.

5 Other Notes & Questions

5.1 Mongoose ODM에 관하여

현재 WRTN 엔지니어링 팀은 Node.js 환경에서 Mongoose ODM을 사용하고 있습니다. Mongoose는 개발 편의성을 제공하지만, 프로덕션 환경에서 아래와 같은 단점을 수반합니다.

첫째, 성능 오버헤드가 발생합니다. Mongoose는 모든 Read 결과에 대해 Document Hydration(getter/setter, virtual, middleware 바인딩)을 수행하며, 벤치마크 기준으로 MongoDB Native Driver 대비 약 2배 느린 Read 성능을 보입니다. populate() 메서드는 내부적으로 참조된 Collection 수만큼 별도의 find() 쿼리를 실행하므로, 한 번의 populate 호출이 3~4회의 네트워크 왕복을 유발할 수 있습니다.

둘째, MongoDB Driver Dependency가 강하게 고정되어 있습니다. Mongoose는 내부적으로 MongoDB Node.js Driver를 래핑하는 구조인데, 특정 Driver 버전에 대해 tilde range(~)로 고정하고 있습니다. 예를 들어, 현재 Mongoose 8.x는 mongodb: ~6.20.0으로 의존하고 있어, MongoDB Driver를 독립적으로 최신 버전(예:

v7.x)으로 업그레이드하는 것이 불가능합니다. 즉, MongoDB Server → MongoDB Driver → Mongoose 순서로 이어지는 업그레이드 체인에서 Mongoose가 병목이 되며, Driver의 신규 기능(Queryable Encryption, CSOT 등)이나 성능 개선, 보안 패치를 적용하려면 Mongoose의 업데이트를 기다려야 합니다. MongoDB Driver의 Major 버전이 올라갈 때마다(v5→v6→v7) Mongoose도 Major 업그레이드가 필요하며, 이 과정에서 Breaking Change 대응 부담이 이중으로 발생합니다.

셋째, MongoDB Technical Support 범위에서 제외됩니다. Mongoose는 Community-Supported Library(유지보수: Automattic)이므로, Mongoose 자체에서 발생하는 이슈는 MongoDB Support Ticket으로 대응받을 수 없으며 커뮤니티에 의존해야 합니다.

단계적으로 Native Driver로 전환하되, 4.1절에서 권고한 messages Collection 분리와 동시에 진행하면 자연스러운 전환 시점이 됩니다. 전환 이전이라도 Read-only 쿼리에 lean() 옵션을 적용하면 Hydration 오버헤드가 제거되어 즉시 Read 성능 개선 효과를 얻을 수 있습니다.

참고: [MongoDB Node.js Driver 공식 문서](https://www.mongodb.com/docs/drivers/node/current/) (https://www.mongodb.com/docs/drivers/node/current/)

5.2 Unique Index 빌드 실패 및 prepareUnique 활용

payment-snapshots Collection에서 originalPaymentId 필드에 대한 Unique Index(originalPaymentId_1) 생성 시도가 기존 중복 데이터로 인해 실패하고 있습니다. Index Build 프로세스가 약 453초(7분 33초) 동안 실행된 후 DuplicateKey 오류와 함께 종료되는 상황이 반복되고 있으며, 이 동안 해당 Collection에 대한 Write Operation에 영향을 줄 수 있습니다.

Unique Index 빌드가 실패하는 근본 원인은 Collection 내에 동일한 originalPaymentId 값을 가진 Document가 이미 존재하기 때문입니다. 전통적인 방법으로는 먼저 모든 중복 데이터를 식별하고 정리한 뒤 Index를 생성해야 하지만, 중복 정리 기간 동안 새로운 중복이 추가로 발생할 수 있다는 문제가 있습니다.

MongoDB 6.0에서 도입된 prepareUnique 옵션은 이 문제를 안전하게 해결할 수 있는 메커니즘을 제공합니다. 이 옵션의 핵심 동작은 다음과 같습니다.

동작	prepareUnique: true 상태	unique: true 상태
기존 중복 데이터	허용 (인덱스 빌드 성공)	존재 시 빌드 실패
새로운 중복 Insert	거부 (DuplicateKey 오류)	거부 (DuplicateKey 오류)
인덱스 타입	Non-unique Index	Unique Index
Application 영향	새 중복만 차단, 기존 쿼리 정상 동작	완전한 Unique 제약

이 기능을 활용하면 서비스 다운타임 없이 안전하게 Unique Index를 구축할 수 있으며, 구체적인 절차는 아래와 같습니다.

1단계로, `prepareUnique` 옵션을 설정하여 인덱스를 빌드합니다. 이 단계에서는 기존 중복 데이터가 있어도 인덱스 빌드가 성공하며, 빌드 완료 시점부터 새로운 중복 Insert는 즉시 차단됩니다.

1단계: `prepareUnique`로 빌드 (기존 중복 허용, 새 중복 차단)

```
db.getCollection("payment-snapshots").createIndex(  
  { originalPaymentId: 1 },  
  { name: "originalPaymentId_1", prepareUnique: true }  
)
```

1단계 완료 후 `$group` Aggregation을 통해 중복 값을 찾아냅니다.

중복 데이터 확인

```
db.getCollection("payment-snapshots").aggregate([  
  { $group: {  
    _id: "$originalPaymentId",  
    count: { $sum: 1 },  
    ids: { $push: "$_id" }  
  }},  
  { $match: { count: { $gt: 1 } } },  
  { $sort: { count: -1 } }  
], { allowDiskUse: true })  
  
// 반환 예시:  
// { _id: "PAY-00123", count: 3, ids: [ObjectId("..."), ObjectId("..."),  
// ObjectId("...")] }  
// { _id: "PAY-00456", count: 2, ids: [ObjectId("..."), ObjectId("...")] }
```

반환된 목록을 기반으로 중복 데이터를 정리합니다. 결제 데이터의 특성상 삭제보다는 비즈니스 로직에 따라 병합하거나 보관용 Collection으로 이동하는 방식이 적절합니다.

2단계로, 모든 중복이 정리된 후 `collMod` 명령으로 인덱스를 Unique로 전환합니다.

2단계: 중복 정리 후 Unique 전환

```
db.runCommand({  
  collMod: "payment-snapshots",  
  index: {  
    keyPattern: { originalPaymentId: 1 },  
    unique: true  
  }  
})
```

Mongoose Schema에서 `unique: true`를 설정하면, Application 시작 시 Mongoose가 자동으로 `ensureIndexes`를 호출하여 해당 Index 생성을 시도합니다. 중복 데이터가 남아있는 상태에서 Application이 재시작되면 동일한 Index 빌드 실패가 매번 반복될 수 있습니다. 따라서 `prepareUnique` 방식으로 수동 관리하는 동안에는 Mongoose Schema에서 `unique` 옵션을 제거하거나, `autoIndex: false` 설정으로 자동 인덱스 생성을 비활성화하는 것을 권장합니다.

5.3 Index를 활용한 In-Memory Sort 제거

MongoDB 로그 분석 중 다수의 쿼리에서 SORT stage가 Explain Plan에 나타나는 것을 확인하였습니다. 쿼리에 `.sort()`가 포함되어 있지만 해당 Sort 패턴을 지원하는 Index가 없으면, MongoDB는 결과를 메모리에 로드한 후 In-Memory Sort를 수행합니다. 이 방식에는 두 가지 핵심 제약이 있습니다.

첫째, In-Memory Sort는 기본적으로 100MB의 메모리 제한이 있으며, 이를 초과하면 쿼리가 실패합니다. `allowDiskUse`를 활성화하면 디스크를 사용하여 제한을 우회할 수 있지만, 디스크 I/O로 인해 성능이 크게 저하됩니다.

둘째, CPU 리소스를 소모합니다. 대량의 Document를 정렬하는 작업은 CPU-intensive한 연산이며, 동시 요청이 많은 환경에서는 전체 쿼리 처리량(Throughput)에 부정적인 영향을 미칩니다.

Index는 이미 키 값 순서대로 정렬된 B-Tree 구조이므로, 쿼리의 Sort 패턴이 Index의 키 순서와 일치하면 MongoDB는 별도의 Sort 단계 없이 Index 순서대로 결과를 반환할 수 있습니다. Explain Plan에서 SORT stage가 사라지고 Index Scan만으로 정렬된 결과를 돌려주는 형태가 됩니다.

가장 효과적인 접근은 ESR(Equality-Sort-Range) Rule을 따르는 Compound Index를 설계하는 것입니다. ESR Rule은 Index 키를 아래 순서로 배치하는 원칙입니다.

순서	구분	설명	예시
1	Equality	등호(=) 조건으로 필터링하는 필드	<code>{ userId: "abc123" }</code>
2	Sort	정렬에 사용되는 필드	<code>.sort({ createdAt: -1 })</code>
3	Range	범위(\$gt, \$lt, \$in 등) 조건 필드	<code>{ status: { \$in: ["active", "pending"] } }</code>

예를 들어, 채팅 메시지를 특정 사용자 기준으로 최신순 조회하는 쿼리가 아래와 같다면:

```
db.messages.find({ chatId: "abc123" })
  .sort({ createdAt: -1 })
  .limit(50)
```

ESR Rule에 따라 `{ chatId: 1, createdAt: -1 }` Compound Index를 생성하면, MongoDB는 Index Scan만으로 `chatId` 필터링과 `createdAt` 역순 정렬을 동시에 처리합니다. Explain Plan에서 SORT stage가 완전히 제거되며, `limit(50)`과 결합하면 Index에서 50건만 읽고 즉시 반환하므로 응답 시간이 크게 단축됩니다.

현재 In-Memory Sort가 발생하고 있는 쿼리를 식별하려면 Atlas Performance Advisor를 확인하거나, 아래와 같이 Explain Plan에서 SORT stage 존재 여부를 점검합니다.

```
db.messages.find({ chatId: "abc123" })
  .sort({ createdAt: -1 })
  .limit(50)
  .explain("executionStats")

// winningPlan에 SORT stage가 있으면 In-Memory Sort 발생
// SORT stage 없이 IXSCAN -> FETCH 순서이면 Index Sort 적용 완료
```

Atlas Performance Advisor에서 Slow Query 탭을 확인하면, Sort가 포함된 고빈도 쿼리에 대해 권장 Index를 자동으로 제안해주므로 이를 참고하시는 것도 좋습니다. 다만, Index 추가는 Write 성능에 영향을 주므로 빈도가 높은 쿼리를 우선 대상으로 선별적으로 적용하는 것을 권장합니다.

5.4 Online Archive를 활용한 데이터 티어링

WRTN 서비스 특성상 채팅 메시지, 사용자 활동 로그, 결제 스냅샷 등 시간이 지남에 따라 접근 빈도가 급격히 낮아지는 데이터가 지속적으로 누적되고 있습니다. 이러한 Cold Data가 Hot Data와 동일한 클러스터에 저장되면 Working Set 크기가 증가하여 RAM 효율이 떨어지고, Index 크기가 비대해지며, 백업 시간과 스토리지 비용이 불필요하게 증가합니다.

Atlas Online Archive는 접근 빈도가 낮은 데이터를 Atlas가 관리하는 Cloud Object Storage(AWS S3, Azure Blob 등)로 자동 이동시키는 기능입니다. 아카이빙된 데이터는 Atlas Data Federation을 통해 단일 Endpoint에서 클러스터 데이터와 함께 쿼리할 수 있으므로, Application 코드 변경 없이 데이터 라이프사이클을 관리할 수 있습니다.

Online Archive는 Atlas UI에서 Archiving Rule을 설정하면 5분 주기로 아카이빙 작업을 자동 수행합니다. Rule 정의 방식은 두 가지입니다.

방식	설명	적합한 Collection 예시
Date Match	특정 Date 필드 + 보관 일수 기준으로 아카이빙	messages (createdAt 기준 90일 이후)
Custom Query	사용자 정의 쿼리 조건으로 아카이빙	payment-snapshots (status: "completed" + 180일 이상)

WRTN 환경에서 Online Archive 우선 적용 대상은 messages Collection입니다. 클러스터 내에서 가장 큰 데이터 볼륨을 차지하고 있으며, 채팅 메시지의 특성상 시간이 지날수록 접근 빈도가 급격히 감소하는 전형적인 Hot-to-Cold 패턴을 보입니다. Date Match 방식으로 createdAt 필드 기준 90일 이상 경과한 메시지를 아카이빙하면, 클러스터의 Active Data 크기가 대폭 축소되어 Working Set 효율, Index 크기, 백업 속도 모두 개선 효과를 기대할 수 있습니다.

마지막으로 Online Archive 도입 시 아래 사항을 사전에 검토하시기 바랍니다.

i Note

첫째, 아카이빙된 데이터는 Read-Only입니다. Cloud Object Storage에 저장된 데이터는 수정이나 삭제가 불가능합니다.

둘째, 아카이브 쿼리는 클러스터 쿼리보다 느립니다. Cloud Object Storage 기반이므로 응답 시간이 수 초 이상 소요될 수 있습니다. 따라서 실시간 서비스 쿼리에는 적합하지 않으며, 관리자 조회, 분쟁 해결, 감사 (Audit) 등 비실시간 접근 용도에 적합합니다.

셋째, Partition 필드 설계가 쿼리 성능을 결정합니다. Online Archive 생성 시 지정하는 Partition 필드(최대 2~3개)는 이후 변경이 불가능하므로, 아카이브 데이터 조회 시 가장 빈번하게 사용할 필드를 신중하게 선택해야 합니다. 예를 들어, messages Collection이라면 chatId와 createdAt을 Partition 필드로 지정하면 특정 채팅방의 과거 메시지 조회 시 최적의 성능을 기대할 수 있습니다.

비용 측면에서, Online Archive는 Cloud Object Storage 요금(GB당 월 단위)과 쿼리 시 처리 데이터량(TB당 \$5.00)으로 과금됩니다. 클러스터 스토리지 비용 대비 상당히 저렴하므로, 데이터가 지속적으로 증가하는 환경에서는 장기적으로 유의미한 비용 절감 효과를 기대할 수 있습니다.

6 Recommended Follow-up Consulting

6.1 Sharding Architecture Optimization

이번 Health Check에서 가장 심각한 이슈로 식별된 shard-0 Hot Shard 문제는, 단순한 설정 변경이 아니라 아키텍처 레벨의 재설계가 필요합니다. 4.1절에서 제시한 두 개의 방식 중 현재 상황에 가장 맞는 방법을 선택하고 실행하기 위해, MongoDB Professional Services가 다음의 영역에서 전문 컨설팅을 제공할 수 있습니다.

단계	컨설팅 내용
Affinity Group 분석	Application 코드 리뷰를 통한 Transaction/\$lookup 의존성 전수 조사. Collection 간 데이터 의존성 맵 작성 및 Affinity Group 정의.
moveCollection 실행 계획	Affinity Group 기반 Collection 배치 전략 수립. shard별 인덱스/데이터 사이즈 균등화 시뮬레이션. 실행 순서 및 롤백 계획 수립.
실행 지원 및 검증	moveCollection 실행 중 모니터링, 성능 변화 실시간 검증, 문제 발생 시 즉시 대응. 완료 후 shard별 워크로드 분포 재검증.

MongoDB 내부에서 수백 건의 Sharding 프로젝트를 수행한 경험과 Best Practice를 기반으로, WRTN Technologies의 특수한 워크로드 패턴에 최적화된 배치 전략을 설계하고 구현하는데 도움을 줄 수 있습니다.

6.2 Query Performance Optimization Workshop

4.2절에서 즉시 적용 가능한 6개 인덱스와 4개 쿼리 패턴 변경을 권고하였으나, 이는 48시간 Slow Query 로그에서 가장 빈번한 항목만 분석한 결과입니다. crack-prod 클러스터에는 645개의 인덱스가 존재하며, 이 중 실제로 사용되지 않는 인덱스가 상당수 포함되어 있을 가능성이 높습니다.

활동	내용	기대 효과
인덱스 전수 조사	\$indexStats를 활용한 645개 인덱스 사용 현황 분석. Hidden Index를 활용한 안전한 미사용 인덱스 정리 절차 수립.	인덱스 사이즈 절감 → WiredTiger Cache 효율 개선 → Disk I/O 감소
추가 Slow Query 분석	4.2절에서 다루지 못한 나머지 Slow Query 패턴 분석. Atlas Performance Advisor 권고 인덱스 검증.	추가 Slow Query 해소, 전체 쿼리 처리량(Throughput) 향상
ESR Rule 실습 워크숍	WRTN 엔지니어링 팀 대상 ESR Rule 기반 인덱스 설계 실습. 실제 crack-prod 쿼리를 활용한 Hands-on 세션.	팀 자체적으로 인덱스 설계/최적화 가능한 역량 내재화

이 워크숍은 단순히 현재 문제를 해결하는 것을 넘어, WRTN 엔지니어링 팀이 향후 새로운 Collection이나 쿼리 패턴이 추가될 때 스스로 최적의 인덱스를 설계할 수 있는 역량을 확보하는 데 중점을 둡니다.

6.3 messages 클러스터 분리 설계

4.1절 Strategy B에서 권고한 messages Collection의 별도 클러스터 분리는 WRTN의 MongoDB 아키텍처를 근본적으로 개선하는 가장 임팩트가 큰 프로젝트입니다. 이 프로젝트는 다음과 같은 전문적인 설계와 실행이 필요합니다.

단계	컨설팅 내용
Design	messages 전용 Sharded Cluster 아키텍처 설계 (Tier, Shard 수, Shard Key 재검토). Application의 messages 접근 패턴 분석 및 Connection 분리 전략 수립. 마이그레이션 계획 수립 (MongoSync 또는 Kafka 기반).
Implement	마이그레이션 실행 지원. 기존 crack-prod에서 messages 관련 Transaction/\$lookup 의존성 제거 검증. Application Connection String 전환 지원.
Validate	마이그레이션 완료 후 데이터 정확성 검증. 새 클러스터와 기존 클러스터의 성능 프로파일링. crack-prod Replica Set 전환 가능성 검토.

6.4 Data Lifecycle Management

5.4절에서 권고한 Online Archive 도입은 messages Collection의 지속적인 데이터 증가에 대한 중장기적 해결책입니다. 현재 4.3B 문서가 계속 증가하고 있어, 아카이빙 전략 없이는 Storage 비용과 인덱스 사이즈가 선형적으로 증가합니다.

영역	컨설팅 내용
아카이빙 전략 설계	messages의 접근 빈도 분석 기반 보관 기간 산정 (90일 vs 180일). Partition 필드(chatId, createdAt) 최적화. 비용 시뮬레이션.
구현 및 검증	Atlas Online Archive 설정, Data Federation Endpoint 구성, 쿼리 성능 검증
확장 적용	payment-snapshots, cracker-histories 등 추가 아카이빙 대상 Collection 식별 및 적용

6.5 Training Subscription

이번 Health Check에서 식별된 이슈들의 상당수는 MongoDB 운영 Best Practice(인덱스 설계, Sharding 전략, 쿼리 최적화)에 대한 깊은 이해가 있었다면 사전에 예방할 수 있었던 항목입니다. WRTN 엔지니어링 팀의 MongoDB 역량을 체계적으로 강화하기 위해 Training Subscription을 권장합니다.

Training Subscription은 1년간 MongoDB의 프리미엄 교육 리소스에 무제한으로 접근할 수 있는 프로그램으로, Instructor-Led Training(실시간 전문가 주도 강의), On-Demand 셀프 학습 라이브러리, Skill Scanner(맞춤형 학습 경로), 그리고 MongoDB Certification 시험 응시권이 포함되어 있습니다.

7개 이상 Subscription을 구매하면 해당 Subscription 등록된 분들을 대상으로 오프라인 워크샵이나 교육을 수행할 수 있습니다.

WRTN Technologies의 현재 상황에 맞춘 권장 학습 경로는 다음과 같습니다.

대상	Day 1	Day 2	Day 3	Day 4
Backend 개발팀	MDB100: Database & Security	MDB200: Optimization & Performance	MDB300: Production Readiness	DEV400: Developer Extension
인프라/DBA 팀	MDB100: Database & Security	MDB200: Optimization & Performance	MDB300: Production Readiness	OFA400: Atlas Admin

위 4일 과정을 완료하면 MongoDB Certification 시험 응시권 1회가 포함되어 있어, 팀원들의 전문성을 공식적으로 입증할 수 있습니다.

추가로 WRTN의 기술 과제에 특화된 워크숍 과정도 권장합니다.

워크숍	기간	관련 Health Check 이슈
MongoDB Internals: Transaction & WiredTiger Deep Dive	3일	Cross-Shard Transaction 이해, WiredTiger Cache 관리
Performance: Schema Validation & Time Series	2일	데이터 모델링 최적화, 쿼리 성능 개선
Architecture: Kafka Integration, Event Driven	2일	CDC Pipeline 최적화, Kafka Connector 운영

Appendix

Appendix1. 미사용 / 중복 인덱스 일람

1GB 이상의 크기를 가진 미사용/중복된 인덱스

Collection	Type	Index Definition	ops	size_mb
wrtn-copy.cracker-histories-temp	Unused	{ createdAt: -1, isConsumed:1 }	0	39321.6
wrtn-copy.cracker-histories-temp	Unused	{ ci:1, type:1, createdAt: -1 }	0	30208
wrtn-copy.cracker-histories-temp	Unused	{ userId:1, isConsumed:1, createdAt: -1 }	0	28979.2
wrtn-copy.cracker-histories	Unused	{ userId:1, type:1, quantity:1, createdAt:1 }	0	28569.6
wrtn-copy.cracker-histories-temp	Redundant	{ userId:1, createdAt: -1 }	0	25292.8
wrtn-copy.cracker-histories-temp	Unused	{ userId:1, type:1, quantity:1, createdAt:1 }	0	21811.2
wrtn-copy.cracker-histories-temp	Redundant	{ userId:1, type:1, createdAt:1 }	0	20787.2
wrtn-copy.cracker-histories	Redundant	{ userId:1, type:1 }	30766	13926.4
wrtn-copy.cracker-histories	Unused	{ userId:1, product:1 }	0	13516.8
wrtn-copy.cracker-histories-temp	Redundant	{ userId:1, type:1 }	0	11264
wrtn-copy.cracker-histories-temp	Unused	{ userId:1, consumedType:1 }	0	11161.6
wrtn-copy.cracker-histories-temp	Unused	{ userId:1, imageGenerationId:1, kind:1 }	0	10854.4
wrtn-copy.cracker-histories-temp	Unused	{ userId:1, product:1 }	0	10649.6
wrtn-copy.chats	Unused	{ messagedAt: -1, createdAt: -1 }	0	2764.8
wrtn-copy.chats	Unused	{ isDeleted:1, userId:1, pinnedAt: -1, messagedAt: -1, createdAt: -1, character.type:1 }	0	2560
wrtn-copy.chat-summaries	Redundant	{ chatId:1, type:1, orderKey: -1 }	6697884	2355.2

Collection	Type	Index Definition	ops	size_mb
wrtn-copy.user-assignments	Redundant	{ userId:1 }	78852	1638.4
wrtn-copy.chats	Redundant	{ userId:1, isDeleted:1 }	0	1433.6